

Báo cáo BTVN 5/5/2026

Telnet Server dùng Multiprocessing

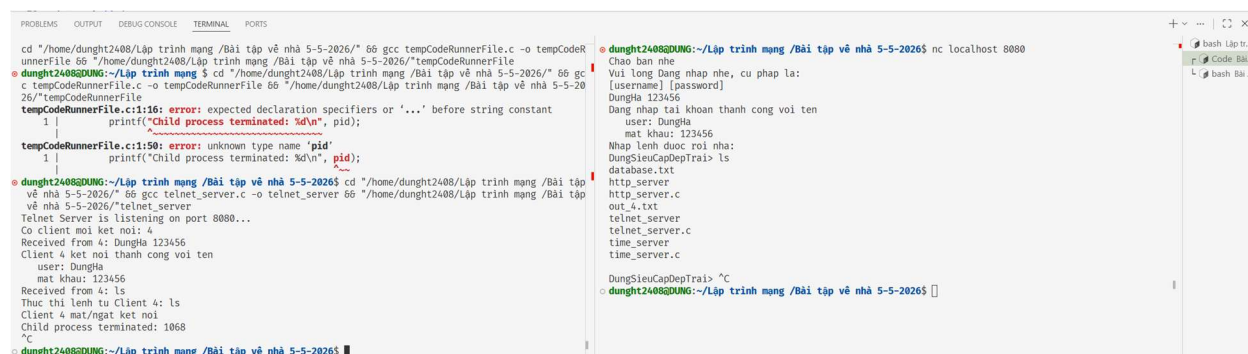
Mô tả cho Bài tập

Chương trình mô phỏng ứng dụng Telnet sử dụng kỹ thuật đa tiến trình cho phép một client điều khiển và thực thi lệnh hệ thống từ xa. Client muốn tham gia điều khiển cần xác thực danh tính đúng cú pháp: **[user] [password]**

Server sẽ lấy thông tin đăng nhập và đối chiếu với cơ sở dữ liệu được lưu trong file *database.txt*. Nếu tài khoản không đúng cú pháp hoặc sai định dạng, Server sẽ báo lỗi và không cho phép thực thi lệnh.

Khi đăng nhập thành công, Client có thể gửi các lệnh hệ thống. Server sẽ tiếp nhận, sử dụng *system()* kết hợp kỹ thuật chuyển hướng luồng (**>out_[client_fd].txt 2>&1**) để thực thi lệnh một cách độc lập, ghi kết quả vào file văn bản tạm thời. Server đọc file này, trả kết quả về màn hình, hiển thị cho Client và dọn dẹp file rác.

Hình ảnh 1.1:



```
cd "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" 66 gcc tempCodeRunnerFile.c -o tempCodeR
unnerFile 66 "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/"tempCodeRunnerFile
dunght2408@DUNG:~/Lập trình mạng $ cd "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" 66 gc
c tempCodeRunnerFile.c -o tempCodeRunnerFile 66 "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-20
26/"tempCodeRunnerFile
tempCodeRunnerFile.c:1:16: error: expected declaration specifiers or '...' before string constant
1 | printf("Child process terminated: %d\n", pid);
  |                ^
tempCodeRunnerFile.c:1:50: error: unknown type name 'pid'
1 | printf("Child process terminated: %d\n", pid);
  |                                         ^
dunght2408@DUNG:~/Lập trình mạng /Bài tập về nhà 5-5-2026$ cd "/home/dunght2408/Lập trình mạng /Bài tập
về nhà 5-5-2026/" 66 gcc telnet_server.c -o telnet_server 66 "/home/dunght2408/Lập trình mạng /Bài tập
về nhà 5-5-2026/"telnet_server
Telnet Server is listening on port 8080...
Go client moi ket noi: 4
Received from 4: DungHa 123456
Client 4 ket noi thanh cong voi ten
user: DungHa
mat khau: 123456
Received from 4: ls
Thuc thi lenh tu Client 4: ls
Client 4 mat/ngat ket noi
Child process terminated: 1068
^C
dunght2408@DUNG:~/Lập trình mạng /Bài tập về nhà 5-5-2026$
```

Hình ảnh 1.2:

Bài tập về nhà 5-5-2026 > database.txt	
1	Dung 1234
2	DungHa 123456

Vì bài này làm nhiều rồi nên em xin phép không trình bày lại ạ.

Time Server

Mô tả cho Bài tập

Ứng dụng server thực hiện chức năng trả về thời gian hiện tại của hệ thống theo định dạng client yêu cầu, sử dụng mô hình đa tiến trình.

Tiến trình con sẽ liên tục nhận dữ liệu từ client. Nếu client gửi lệnh đúng cú pháp GET_TIME <định_dạng> (hỗ trợ các định dạng: dd/mm/yyyy, dd/mm/yy, mm/dd/yyyy, mm/dd/yy), server sẽ gọi hàm strftime để map chuỗi định dạng tương ứng và gửi trả lại thời gian thực. Nếu sai lệnh hoặc sai định dạng, server gửi thông báo lỗi yêu cầu nhập lại. Khi client ngắt kết nối, tiến trình con gọi exit(0). Để tránh hiện tượng zombie process, tiến trình cha sử dụng cơ chế signal(SIGCHLD, signal_handler) kết hợp hàm wait(NULL) để tự động thu hồi tài nguyên ngay khi tiến trình con kết thúc.

Hình ảnh 2.1:



```
cd "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" 66 gcc time_server.c -o time_server 66 "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" time_server
dunght2408@DUNG:~/Lập trình mạng $ cd "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" 66 gcc time_server.c -o time_server 66 "/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" time_server
Time Server is listening on port 8080...
Có kết nối mới 4
Client 4 mất/ngắt kết nối
Child process terminated: 3936
^C
dunght2408@DUNG:~/Lập trình mạng /Bài tập về nhà 5-5-2026$
```

```
dunght2408@DUNG:~/Lập trình mạng $ nc localhost 8080
GET Time dd/mm/yy
Lỗi CMD, Nhập lại đi!
GETTIMEdd/mm/yy
Lỗi Lệnh, Nhập lại đi!
GET_TIME d/mm/yy
Lỗi Format, Nhập lại đi!
GET_TIME mm/dd/yy
05/08/26
^C
dunght2408@DUNG:~/Lập trình mạng $
```

Dựa vào kết quả chạy, ta thấy chương trình đã hoạt động đúng theo mục tiêu của mô hình. Server lắng nghe kết nối, tạo tiến trình con, kiểm tra tính đúng đắn của câu lệnh và trả về thời gian theo chuẩn định dạng.

Http Server dùng Multiprocessing

Mô tả cho Bài tập

Ứng dụng server thực hiện chức năng phản hồi HTTP cơ bản sử dụng mô hình pre-forking. Khi khởi động, server sẽ tạo sẵn một pool gồm 4 tiến trình con (worker processes) thông qua hàm fork() và chờ các kết nối TCP từ client ở cổng 8080. Khi có một HTTP request gửi đến, một tiến trình con đang rảnh rỗi sẽ tiếp nhận kết nối qua hàm accept(), in ra màn hình terminal PID của nó để xác nhận đang xử lý, sau đó phản hồi lại cho client một thông báo HTTP 200 OK kèm theo nội dung HTML tĩnh có chứa thẻ h1. Quá trình này là phi trạng thái (stateless), tiến trình sẽ lập tức đóng kết nối ngay sau khi gửi phản hồi xong và quay trở lại vòng lặp chờ request mới.

Hình ảnh 3.1:



```
cd ~/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" 66 gcc http_server.c -o http_server 66
/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/"http_server
dunght2408@DUNG:~/Lập trình mạng $ cd ~/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/" 66 gcc
http_server.c -o http_server 66 ~/home/dunght2408/Lập trình mạng /Bài tập về nhà 5-5-2026/"http_server
F
Preforking HTTP Server đang chạy trên cổng 8080 với 4 tiến trình...
Process PID 2228 đang xử lý request
^C
dunght2408@DUNG:~/Lập trình mạng /Bài tập về nhà 5-5-2026$

dunght2408@DUNG:~/Lập trình mạng $ nc localhost 8080
Hello
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Connection: close

<html><body><h1>DungSieuCapDepTrai HTTP Server</h1></body></html>
dunght2408@DUNG:~/Lập trình mạng $
```

Dựa vào kết quả chạy, ta thấy chương trình đã hoạt động đúng theo mục tiêu của mô hình pre-forking. Ban đầu, tiến trình cha sinh ra 4 tiến trình con cùng chia sẻ chung một socket và cùng chờ cấp phát tại hàm accept(). Khi client (trình duyệt web) truy cập vào địa chỉ localhost:8080, hệ điều hành điều phối để giao kết nối này cho một worker đang rảnh rỗi xử lý. Ở phía client, trình duyệt cũng nhận và render thành công giao diện với dòng chữ "DungSieuCapDepTrai HTTP Server", cho thấy gói tin HTTP Response đã được truyền gửi chính xác.